

Exercícios de Lógica de Programação com C#

Material para o aluno — C# Console → Web API

Objetivo: resolver primeiro cada problema em C# tradicional e, posteriormente, transformar a solução em um endpoint de ASP.NET Core Web API.

Orientação: utilize estruturas fundamentais de programação. Evite LINQ, lambda, operador ternário e outras abstrações que escondam a lógica. A solução deve deixar explícito o raciocínio do algoritmo.

Etapa 1	Etapa 2
Exercícios 1 a 5	Exercícios 6 a 10
Decisão, operadores relacionais e lógicos	Vetores e estruturas de repetição

Exercício 1 — Verificação de Aprovação em Disciplina

Conteúdos: Estruturas de decisão, operadores relacionais e operadores lógicos.

Enunciado

Crie um programa que receba duas notas e a quantidade de faltas de um aluno. Calcule a média e determine a situação. O aluno será Aprovado se a média for maior ou igual a 7 e as faltas forem menores ou iguais a 10. Será Reprovado por nota se a média for menor que 7 e as faltas forem menores ou iguais a 10. Caso tenha mais de 10 faltas, será Reprovado por falta.

Resposta esperada no Console

```
Informe a primeira nota: 8  
Informe a segunda nota: 7  
Informe o número de faltas: 5
```

```
Média: 7,00  
Situação: Aprovado
```

Desafio de conversão para Web API

Depois de concluir a solução em C#, transforme-a em uma ASP.NET Core Web API. Preserve a lógica desenvolvida e escolha corretamente o método HTTP e a forma de entrada dos dados.

Endpoint sugerido: POST /api/alunos/avaliacao

Exercício 2 — Classificação de Faixa Etária

Conteúdos: Estruturas de decisão e operadores relacionais.

Enunciado

Receba a idade de uma pessoa e classifique-a como Criança (0 a 12), Adolescente (13 a 17), Adulto (18 a 59) ou Idoso (60 ou mais). Informe também a situação do voto: não pode votar, voto opcional ou voto obrigatório.

Resposta esperada no Console

```
Informe a idade: 25
```

```
Classificação: Adulto
```

```
Situação do voto: Voto obrigatório
```

Desafio de conversão para Web API

Depois de concluir a solução em C#, transforme-a em uma ASP.NET Core Web API. Preserve a lógica desenvolvida e escolha corretamente o método HTTP e a forma de entrada dos dados.

Endpoint sugerido: GET /api/pessoas/classificacao/{idade}

Exercício 3 — Validação de Acesso

Conteúdos: Operadores lógicos, relacionais e valores booleanos.

Enunciado

Crie um programa que receba a idade, a senha e a situação da conta. O acesso somente será permitido se a pessoa tiver 18 anos ou mais, a senha for correta e a conta estiver ativa.

Resposta esperada no Console

```
Informe a idade: 25  
Informe a senha: 1234  
A conta está ativa? true
```

```
Acesso permitido.
```

Desafio de conversão para Web API

Depois de concluir a solução em C#, transforme-a em uma ASP.NET Core Web API. Preserve a lógica desenvolvida e escolha corretamente o método HTTP e a forma de entrada dos dados.

Endpoint sugerido: POST /api/acesso/validar

Exercício 4 — Cálculo de Salário com Bonificação

Conteúdos: Decisões encadeadas, operadores relacionais e cálculo.

Enunciado

Receba o salário e o tempo de empresa de um funcionário. Salário inferior a R\$ 2.000 recebe 20% de bonificação; entre R\$ 2.000 e R\$ 5.000 recebe 10%; acima de R\$ 5.000 recebe 5%. Funcionários com mais de 10 anos de empresa recebem mais 5%.

Resposta esperada no Console

```
Informe o salário: 4000
Informe os anos de empresa: 12

Percentual de bonificação: 15%
Valor da bonificação: R$ 600,00
Salário final: R$ 4600,00
```

Desafio de conversão para Web API

Depois de concluir a solução em C#, transforme-a em uma ASP.NET Core Web API. Preserve a lógica desenvolvida e escolha corretamente o método HTTP e a forma de entrada dos dados.

Endpoint sugerido: POST /api/funcionarios/bonificacao

Exercício 5 — Sistema de Desconto em Loja

Conteúdos: Decisões, operadores lógicos e cálculo.

Enunciado

Uma loja concede 15% de desconto para compras acima de R\$ 1.000, 10% para compras entre R\$ 500 e R\$ 1.000 e 5% para compras abaixo de R\$ 500. Clientes cadastrados recebem mais 5%. Calcule o valor do desconto e o valor final.

Resposta esperada no Console

```
Informe o valor da compra: 800  
Cliente cadastrado? true
```

```
Percentual de desconto: 15%  
Valor do desconto: R$ 120,00  
Valor final: R$ 680,00
```

Desafio de conversão para Web API

Depois de concluir a solução em C#, transforme-a em uma ASP.NET Core Web API. Preserve a lógica desenvolvida e escolha corretamente o método HTTP e a forma de entrada dos dados.

Endpoint sugerido: POST /api/vendas/calcular

Exercício 6 — Leitura e Exibição de Vetor

Conteúdos: Vetores e estrutura de repetição for.

Enunciado

Leia 10 números inteiros e armazene-os em um vetor. Depois, exiba todos os números na ordem em que foram informados e, em seguida, na ordem inversa.

Resposta esperada no Console

```
Informe os 10 números:
```

```
10  
20  
30  
40  
50  
60  
70  
80  
90  
100
```

```
Ordem original:
```

```
10 20 30 40 50 60 70 80 90 100
```

```
Ordem inversa:
```

```
100 90 80 70 60 50 40 30 20 10
```

Desafio de conversão para Web API

Depois de concluir a solução em C#, transforme-a em uma ASP.NET Core Web API. Preserve a lógica desenvolvida e escolha corretamente o método HTTP e a forma de entrada dos dados.

Endpoint sugerido: POST /api/numeros/inverter

Exercício 7 — Soma e Média de um Vetor

Conteúdos: Vetores, for, acumulador, maior e menor.

Enunciado

Leia 10 números reais e armazene-os em um vetor. Calcule a soma, a média, o maior valor e o menor valor.

Resposta esperada no Console

```
Informe os 10 números:
```

```
10
```

```
5
```

```
8
```

```
20
```

```
15
```

```
7
```

```
12
```

```
3
```

```
18
```

```
2
```

```
Soma: 100
```

```
Média: 10,00
```

```
Maior: 20
```

```
Menor: 2
```

Desafio de conversão para Web API

Depois de concluir a solução em C#, transforme-a em uma ASP.NET Core Web API. Preserve a lógica desenvolvida e escolha corretamente o método HTTP e a forma de entrada dos dados.

Endpoint sugerido: POST /api/numeros/estatisticas

Exercício 8 — Contagem de Pares e Ímpares

Conteúdos: Vetores, repetição, operador módulo e acumuladores.

Enunciado

Leia 10 números inteiros e armazene-os em um vetor. Informe a quantidade de números pares e ímpares, além da soma dos pares e da soma dos ímpares.

Resposta esperada no Console

```
Informe os 10 números:
```

```
2  
5  
8  
7  
10  
3  
12  
9  
4  
6
```

```
Quantidade de pares: 6
```

```
Quantidade de ímpares: 4
```

```
Soma dos pares: 42
```

```
Soma dos ímpares: 24
```

Desafio de conversão para Web API

Depois de concluir a solução em C#, transforme-a em uma ASP.NET Core Web API. Preserve a lógica desenvolvida e escolha corretamente o método HTTP e a forma de entrada dos dados.

Endpoint sugerido: POST /api/numeros/pares-impares

Exercício 9 — Pesquisa de um Número no Vetor

Conteúdos: Vetores, repetição, pesquisa e contador.

Enunciado

Leia 10 números inteiros e armazene-os em um vetor. Depois solicite um número para pesquisa. Informe se ele foi encontrado, em quais posições aparece e quantas vezes aparece.

Resposta esperada no Console

```
Vetor:
10 20 30 20 40 50 20 60 70 80

Número para pesquisar: 20

Número encontrado.
Quantidade de ocorrências: 3
Posições: 1, 3 e 6
```

Desafio de conversão para Web API

Depois de concluir a solução em C#, transforme-a em uma ASP.NET Core Web API. Preserve a lógica desenvolvida e escolha corretamente o método HTTP e a forma de entrada dos dados.

Endpoint sugerido: POST /api/numeros/pesquisar

Exercício 10 — Classificação de Positivos e Negativos

Conteúdos: Vetores, repetição, decisões e acumuladores.

Enunciado

Leia 10 números inteiros e armazene-os em um vetor. Informe a quantidade de positivos, negativos e zeros, além da soma dos positivos e da soma dos negativos.

Resposta esperada no Console

```
Informe os 10 números:
```

```
10
```

```
-5
```

```
0
```

```
8
```

```
-2
```

```
4
```

```
0
```

```
-7
```

```
6
```

```
3
```

```
Quantidade de positivos: 5
```

```
Quantidade de negativos: 3
```

```
Quantidade de zeros: 2
```

```
Soma dos positivos: 31
```

```
Soma dos negativos: -14
```

Desafio de conversão para Web API

Depois de concluir a solução em C#, transforme-a em uma ASP.NET Core Web API. Preserve a lógica desenvolvida e escolha corretamente o método HTTP e a forma de entrada dos dados.

Endpoint sugerido: POST /api/numeros/classificar